

AI Assistant Coding

Assignment 10.1

Name : E.ANJANI

HT. No : 2303A52419

Batch: 32

Task Description #1 – Syntax and Logic Errors

Task: Use AI to identify and fix syntax and logic errors in a faulty Python script.

Sample Input Code: # Calculate average score of a student

```
def calc_average(marks):
```

```
    total = 0
```

```
    for m in marks:
```

```
        total += m
```

```
    average = total / len(marks)
```

```
    return avrage # Typo here
```

```
marks = [85, 90, 78, 92]
```

```
print("Average Score is ", calc_average(marks))
```

Expected Output:

- Corrected and runnable Python code with explanations of the fixes.

Code:

```
Task1.py > ...
1 # refactor the below code to fix the errors to get runnable code with explanation
2 def calc_average(marks):
3     total = 0
4     for m in marks:
5         total += m
6         average = total / len(marks)
7     return average # Typo here fixed
8
9 marks = [85, 90, 78, 92]
10 print("Average Score is ", calc_average(marks))
```

Output:

```
PS D:\courses\AIAC\Lab_ass_10.1> & c:\python314\python.exe d:/courses/AIAC/Lab_ass_10.1/Task1.py
● Average Score is 86.25
○ PS D:\courses\AIAC\Lab_ass_10.1> |
```

Task Description #2 – PEP 8 Compliance

Task: Use AI to refactor Python code to follow PEP 8 style guidelines.

Sample Input Code:

```
def area_of_rect(L,B) : return L*B

    print(area_of_rect(10,20))
```

Expected Output:

- Well-formatted PEP 8-compliant Python code.

Code:

```
demo.py / ...
1  def area_of_rectangle(L,B):return L*B
2  print(area_of_rectangle(5,10))
3  #refactor the above code with proper naming conventions and formatting also include a docs
4  #add type hints to the function parameters and return type.
5  def calculate_rectangle_area(length: float, breadth: float) -> float:
6      """
7      Calculate the area of a rectangle given its length and breadth.
8
9      Parameters:
10     length (float): The length of the rectangle.
11     breadth (float): The breadth of the rectangle.
12
13     Returns:
14     float: The area of the rectangle.
15
16     Raises:
17     ValueError: If length or breadth is negative.
18     TypeError: If length or breadth is not a number.
19     """
20     if length < 0 or breadth < 0:
21         raise ValueError("Length and breadth must be non-negative.")
22     if not isinstance(length, (int, float)) or not isinstance(breadth, (int, float)):
23         raise TypeError("Length and breadth must be numbers.")
24     return length * breadth
25 print(calculate_rectangle_area(5, 10))

PS D:\courses\AIAC\Lab_ass_10.1> & c:\python314\python.exe d:/courses/AIAC/Lab_ass_10.1/demo.py
50
50
PS D:\courses\AIAC\Lab_ass_10.1> |
```

Task Description #3 – Readability Enhancement

Task: Use AI to make code more readable without changing its logic.

- Python code with descriptive variable names, inline comments, and clear formatting.

Code:

```
Task3.py > ...
1 #refactor the below code with proper variable names, inline comments and clear formatting
2 def calculate_percentage(amount, percentage):
3     """Calculate the percentage of a given amount."""
4     return amount * percentage / 100
5
6 amount = 200
7 percentage = 15
8 print(calculate_percentage(amount, percentage))

PS D:\courses\AIAC\Lab_ass_10.1> & c:\python314\python.exe d:/courses/AIAC/Lab_ass_10.1/Task3.py
30.0
PS D:\courses\AIAC\Lab_ass_10.1> |
```

Task Description #4 – Refactoring for Maintainability

Task: Use AI to break repetitive or long code into reusable functions.

Sample Input Code:

```
students = ["Alice", "Bob", "Charlie"]
print("Welcome", students[0])
print("Welcome", students[1])
print("Welcome", students[2])
```

Expected Output:

- Modular code with reusable functions.

Code:

```
Task4.py > ...
1 #refactor the below code with proper reusable functions
2 students = ["Alice", "Bob", "Charlie"]
3
4 def welcome_student(student_name):
5     print("Welcome", student_name)
6
7 for student in students:
8     welcome_student(student)

PS D:\courses\AIAC\Lab_ass_10.1> & c:\python314\python.exe d:/courses/AIAC/Lab_ass_10.1/Task4.py
Welcome Alice
Welcome Bob
Welcome Charlie
PS D:\courses\AIAC\Lab ass 10.1> |
```

Task Description #5 – Performance Optimization

Task: Use AI to make the code run faster.

Sample Input Code: # Find squares of numbers

```
nums = [i for i in range(1,1000000)]
start_time = time.time()
squares = []
for n in nums:
    squares.append(n**2)
```

```
print(len(squares))
end_time = time.time()
print(f"Time taken: {end_time - start_time} seconds")
```

Expected Output:

- Optimized code using list comprehensions or vectorized operations.

CODE:

```
Task5.py > ...
1 import time
2 time1=time.time()
3 nums = [i for i in range(1,1000000)]
4 squares = []
5 for n in nums:
6     squares.append(n**2)
7 print(len(squares))
8 time2 =time.time()
9 print("Time taken:", time2-time1)
10 #refactor above code to reduce time complexity
11 time3 = time.time()
12 squares = [n**2 for n in range(1, 1000000)]
13 print(len(squares))
14 time4 = time.time()
15 print("Time taken:", time4-time3)

PS D:\courses\AIAC\Lab_ass_10.1> & c:\python314\python.exe d:/courses/AIAC/Lab_ass_10.1/Task5.py
999999
Time taken: 0.13776731491088867
999999
Time taken: 0.07486605644226074
PS D:\courses\AIAC\Lab ass 10.1>
```

Task Description #6 – Complexity Reduction

Task: Use AI to simplify overly complex logic.

Sample Input Code:

```
def grade(score):
    if score >= 90:
        return "A"
    else:
        if score >= 80:
            return "B"
        else:
            if score > 70:
                return "C"
            else:
                if score > 60:
                    return "D"
                else:
                    return "F"
```

Expected Output:

- Cleaner logic using elif or dictionary mapping.

Code:

```
1 # refactor the below code with proper indentation and elif logics
2 def grade(score):
3     if score >= 90:
4         | return "A"
5     elif score >= 80:
6         | return "B"
7     elif score >= 70:
8         | return "C"
9     elif score >= 60:
10        | return "D"
11    else:
12        | return "F"
13    # Example
14    print(grade(85))
```

PS D:\courses\AIAC\Lab_ass_10.1> & c:\python314\python.exe d:/courses/AIAC/Lab_ass_10.1/Task6.py
B
PS D:\courses\AIAC\Lab ass 10.1>